

```

1 package Polymorphism;
2
3 /**
4  * SUPERCLASS (Parent Class): Kapal
5  *
6  * Class ini menjadi fondasi/induk untuk semua tipe kapal di wilayah pelabuhan.
7  * Di dalam kelas ini disiapkan:
8  * 1. Method yang akan di-OVERRIDE oleh subclass (Method Overriding).
9  * 2. Method dengan beberapa variasi parameter yang sama (Method Overloading).
10 */
11 public class Kapal {
12     // Atribut yang dimiliki oleh semua jenis kapal
13     private String namaKapal;
14     private String nomorRegistrasi;
15     private double kapasitasTon;
16
17     // Constructor Superclass
18     public Kapal(String namaKapal, String nomorRegistrasi, double kapasitasTon) {
19         this.namaKapal = namaKapal;
20         this.nomorRegistrasi = nomorRegistrasi;
21         this.kapasitasTon = kapasitasTon;
22     }
23
24     // Getter untuk mengakses atribut private
25     public String getNamaKapal() {
26         return namaKapal;
27     }
28
29     public String getNomorRegistrasi() {
30         return nomorRegistrasi;
31     }
32
33     public double getKapasitasTon() {
34         return kapasitasTon;
35     }
36
37     // =====
38     // 1. CONTOH METHOD OVERRIDING (Polimorfisme Dinamis / Runtime)
39     // =====
40     // Method di bawah ini merupakan method bawaan dari Superclass.
41     // Subclass (KapalKargo, KapalTanker, KapalPenumpang) akan menimpa/mengganti
42     // (OVERRIDE) isi dari method ini sesuai dengan logika khusus masing-masing kapal.
43     // =====
44
45     /**
46      * Menghitung biaya tambat/berlabuh dasar di pelabuhan per hari.
47      * @param jumlahHari Durasi berlabuh dalam hari.
48      * @return Biaya tambat dasar.
49      */
50     public double hitungBiayaTambat(int jumlahHari) {
51         // Biaya dasar standar pelabuhan: Rp 500.000 / hari
52         return jumlahHari * 500000.0;
53     }
54
55     /**
56      * Menjelaskan prosedur proses bongkar muat kapal secara umum.
57      */
58     public void prosesBongkarMuat() {
59         System.out.println("Kapal " + namaKapal + " sedang melakukan proses bongkar muat standar pelabuhan.");
60     }
61
62     // =====
63     // 2. CONTOH METHOD OVERLOADING (Polimorfisme Statis / Compile-time)
64     // =====
65     // Method di bawah ini memiliki NAMA YANG SAMA ('cetakManifest'),
66     // tetapi memiliki JUMLAH / TIPE PARAMETER YANG BERBEDA.
67     // Java menentukan method mana yang dijalankan berdasarkan argumen yang dikirim saat pemanggilan.
68     // =====
69
70     /**
71      * Overload 1: Cetak manifest kapal tanpa parameter tambahan.
72      */
73     public void cetakManifest() {
74         System.out.println("--- MANIFEST KAPAL ---");
75         System.out.println("Nama Kapal      : " + namaKapal);
76         System.out.println("No. Registrasi   : " + nomorRegistrasi);
77         System.out.println("Kapasitas (Ton)  : " + kapasitasTon + " Ton");
78     }
79
80     /**
81      * Overload 2: Cetak manifest dengan 1 parameter (Nama Kapten).
82      * @param namaKapten Nama kapten yang memimpin kapal.
83      */
84     public void cetakManifest(String namaKapten) {
85         cetakManifest(); // Memanggil Overload 1 agar tidak redundan
86         System.out.println("Kapten Kapal    : " + namaKapten);
87     }
88
89     /**
90      * Overload 3: Cetak manifest dengan 2 parameter (Nama Kapten & Jumlah ABK).
91      * @param namaKapten Nama kapten yang memimpin kapal.
92      * @param jumlahABK Jumlah Anak Buah Kapal (kru).
93      */
94     public void cetakManifest(String namaKapten, int jumlahABK) {
95         cetakManifest(namaKapten); // Memanggil Overload 2 agar tidak redundan
96         System.out.println("Jumlah ABK      : " + jumlahABK + " orang");
97     }
98 }
99

```

```

1 package Polymorphism;
2
3 /**
4  * SUBCLASS 1: KapalKargo
5  *
6  * Kelas ini mewarisi (extends) dari kelas induk 'Kapal'.
7  * Menambahkan atribut khusus 'jumlahKontainer' dan melakukan OVERRIDING
8  * pada method 'hitungBiayaTambat' dan 'prosesBongkarMuat'.
9  */
10 public class KapalKargo extends Kapal {
11     private int jumlahKontainer;
12
13     // Constructor Subclass
14     public KapalKargo(String namaKapal, String nomorRegistrasi, double kapasitasTon, int jumlahKontainer) {
15         // Memanggil constructor dari Superclass (Kapal) menggunakan super(...)
16         super(namaKapal, nomorRegistrasi, kapasitasTon);
17         this.jumlahKontainer = jumlahKontainer;
18     }
19
20     // =====
21     // METHOD OVERRIDING (Polimorfisme Dinamis)
22     // =====
23     // Anotasi @Override menandakan bahwa method ini menggantikan/menimpa
24     // method dengan nama dan parameter yang sama di Superclass (Kapal).
25     // =====
26
27     /**
28     * OVERRIDE 1: Menghitung biaya tambat khusus Kapal Kargo.
29     * Rumus: Biaya dasar tambat (dari super.hitungBiayaTambat) + sewa crane per kontainer.
30     */
31     @Override
32     public double hitungBiayaTambat(int jumlahHari) {
33         double biayaDasar = super.hitungBiayaTambat(jumlahHari); // Memanggil logika biaya dasar dari kelas Kapal
34         double biayaKontainer = jumlahKontainer * 50000.0; // Tambahan Rp 50.000 per kontainer
35         return biayaDasar + biayaKontainer;
36     }
37
38     /**
39     * OVERRIDE 2: Menjelaskan proses bongkar muat khusus Kapal Kargo (menggunakan Gantry Crane).
40     */
41     @Override
42     public void prosesBongkarMuat() {
43         System.out.println("Kapal Kargo [" + getNamaKapal() + "] mengoperasikan derek pelabuhan (Gantry Crane) untuk memindahkan "
44             + jumlahKontainer + " kontainer ke area penumpukan (Yard).");
45     }
46 }
47

```

```

1 package Polymorphism;
2
3 /**
4  * SUBCLASS 3: KapalPenumpang
5  *
6  * Kelas ini mewarisi (extends) dari kelas induk 'Kapal'.
7  * Menambahkan atribut khusus 'jumlahPenumpang' dan melakukan OVERRIDING
8  * pada method 'hitungBiayaTambat' dan 'prosesBongkarMuat'.
9  */
10 public class KapalPenumpang extends Kapal {
11     private int jumlahPenumpang;
12
13     // Constructor Subclass
14     public KapalPenumpang(String namaKapal, String nomorRegistrasi, double kapasitasTon, int jumlahPenumpang) {
15         // Memanggil constructor dari Superclass (Kapal) menggunakan super(...)
16         super(namaKapal, nomorRegistrasi, kapasitasTon);
17         this.jumlahPenumpang = jumlahPenumpang;
18     }
19
20     // =====
21     // METHOD OVERRIDING (Polimorfisme Dinamis)
22     // =====
23
24     /**
25     * OVERRIDE 1: Menghitung biaya tambat khusus Kapal Penumpang.
26     * Rumus: Biaya dasar tambat + Pas Pelabuhan per penumpang.
27     */
28     @Override
29     public double hitungBiayaTambat(int jumlahHari) {
30         double biayaDasar = super.hitungBiayaTambat(jumlahHari);
31         double pasPelabuhan = jumlahPenumpang * 10000.0; // Pas pelabuhan Rp 10.000 per orang
32         return biayaDasar + pasPelabuhan;
33     }
34
35     /**
36     * OVERRIDE 2: Menjelaskan proses bongkar muat khusus Kapal Penumpang (menggunakan Garbarata).
37     */
38     @Override
39     public void prosesBongkarMuat() {
40         System.out.println("Kapal Penumpang [" + getNamaKapal() + "] membuka garbarata/jembatan dermaga untuk menurunkan "
41             + jumlahPenumpang + " penumpang dan kendaraan penumpang.");
42     }
43 }
44

```

```

1  package Polymorphism;
2
3  /**
4   * SUBCLASS 2: KapalTanker
5   *
6   * Kelas ini mewarisi (extends) dari kelas induk 'Kapal'.
7   * Menambahkan atribut khusus 'volumeliters' (muatan cair/minyak) dan melakukan OVERRIDING
8   * pada method 'hitungBiayaTambat' dan 'prosesBongkarMuat'.
9   */
10 public class KapalTanker extends Kapal {
11     private double volumeliters;
12
13     // Constructor Subclass
14     public KapalTanker(String namaKapal, String nomorRegistrasi, double kapasitasTon, double volumeliters) {
15         // Memanggil constructor dari Superclass (Kapal) menggunakan super(...)
16         super(namaKapal, nomorRegistrasi, kapasitasTon);
17         this.volumeliters = volumeliters;
18     }
19
20     // =====
21     // METHOD OVERRIDING (Polimorfisme Dinamis)
22     // =====
23
24     /**
25     * OVERRIDE 1: Menghitung biaya tambat khusus Kapal Tanker.
26     * Rumus: Biaya dasar tambat + Biaya penanganan risiko bahan berbahaya (Hazard Safety Fee).
27     */
28     @Override
29     public double hitungBiayaTambat(int jumlahHari) {
30         double biayaDasar = super.hitungBiayaTambat(jumlahHari);
31         double biayaHazard = 2000000.0; // Biaya keamanan bahan berbahaya Rp 2.000.000
32         return biayaDasar + biayaHazard;
33     }
34
35     /**
36     * OVERRIDE 2: Menjelaskan proses bongkar muat khusus Kapal Tanker (menggunakan pompa pipa cair).
37     */
38     @Override
39     public void prosesBongkarMuat() {
40         System.out.println("Kapal Tanker [" + getNamaKapal() + "] menghubungkan pipa penyalur keselamatan tinggi untuk menyedot "
41             + String.format("%.0f", volumeliters) + " liter bahan bakar ke tangki penyimpanan dermaga.");
42     }
43 }
44

```

```

1 package Polymorphism;
2
3 /**
4  * MAIN CLASS: MainRoot
5  *
6  * Kelas utama untuk menjalankan dan menguji penerapan konsep Polimorfisme:
7  * 1. Dynamic Polymorphism (Method Overriding): Variabel bertipe Superclass (Kapal)
8  *    dapat menampung berbagai objek dari Subclass (KapalKargo, KapalTanker, KapalPenumpang).
9  * 2. Static Polymorphism (Method Overloading): Pemanggilan method 'cetakManifest'
10 *    dengan variasi parameter yang berbeda-beda.
11 */
12 public class MainRoot {
13     public static void main(String[] args) {
14         System.out.println("=====");
15         System.out.println("    SISTEM MANAJEMEN OPERASIONAL PELABUHAN ADRESTEIA");
16         System.out.println("=====\\n");
17
18         // -----
19         // 1. DEMO METHOD OVERRIDING (Polimorfisme Dinamis / Runtime)
20         // -----
21         // PERHATIKAN: Tipe variabel adalah 'Kapal' (Superclass),
22         // tetapi objek yang dibuat adalah instansi Subclass masing-masing.
23         Kapal kargo = new KapalKargo("MV Ocean Express", "KG-8891", 15000.0, 120);
24         Kapal tanker = new KapalTanker("MT Pertamina Hero", "TK-4021", 25000.0, 5000000.0);
25         Kapal penumpang = new KapalPenumpang("KMP Dharma Rencana", "PN-1044", 5000.0, 450);
26
27         System.out.println("=== 1. PROSES BONGKAR MUAT (METHOD OVERRIDING) ===");
28         // Walaupun dipanggil melalui tipe Kapal, Java secara otomatis mengeksekusi
29         // method prosesBongkarMuat() versi subclass masing-masing di runtime!
30         kargo.prosesBongkarMuat();
31         tanker.prosesBongkarMuat();
32         penumpang.prosesBongkarMuat();
33
34         System.out.println("\\n=== 2. HITUNG BIAYA TAMBAT / BERLABUH (OVERRIDING - 3 HARI) ===");
35         // Method hitungBiayaTambat() yang dipanggil adalah versi override dari masing-masing subclass
36         System.out.println("Biaya Tambat " + kargo.getNamaKapal() + " : Rp " + String.format("%.0f", kargo.hitungBiayaTambat(3)));
37         System.out.println("Biaya Tambat " + tanker.getNamaKapal() + " : Rp " + String.format("%.0f", tanker.hitungBiayaTambat(3)));
38         System.out.println("Biaya Tambat " + penumpang.getNamaKapal() + " : Rp " + String.format("%.0f", penumpang.hitungBiayaTambat(3)));
39
40         System.out.println("\\n-----\\n");
41
42         // -----
43         // 2. DEMO METHOD OVERLOADING (Polimorfisme Statis / Compile-time)
44         // -----
45         System.out.println("=== 3. CETAK MANIFEST KAPAL (METHOD OVERLOADING) ===");
46
47         // Overload 1: Memanggil cetakManifest() tanpa parameter
48         System.out.println("\\n[Format 1: Tanpa Parameter]");
49         kargo.cetakManifest();
50
51         // Overload 2: Memanggil cetakManifest(String) dengan 1 parameter (nama kapten)
52         System.out.println("\\n[Format 2: Dengan Nama Kapten]");
53         tanker.cetakManifest("Capt. Hendra Wijaya");
54
55         // Overload 3: Memanggil cetakManifest(String, int) dengan 2 parameter (nama kapten & jumlah ABK)
56         System.out.println("\\n[Format 3: Dengan Nama Kapten & Jumlah ABK]");
57         penumpang.cetakManifest("Capt. Agus Santoso", 35);
58
59         System.out.println("\\n=====");
60     }
61 }
62

```